

PRODUTO API

Documentação Técnica Completa

ASP.NET Core 8 · MySQL · Arquitetura em Camadas

Versão 1.0

1. Visão Geral

O Produto API é uma Web API REST desenvolvida com ASP.NET Core 8 para gerenciamento completo de produtos. O sistema utiliza arquitetura em camadas com separação clara de responsabilidades, conexão com banco de dados MySQL e documentação interativa via Swagger.

1.1 Tecnologias Utilizadas

- ASP.NET Core 8 (Web API)
- C# com .NET 8
- MySQL 8.0 via MySql.Data
- Swagger (Swashbuckle.AspNetCore 6.9)
- LINQ para consultas e cálculos

2. Arquitetura do Projeto

O projeto segue o padrão Controller → Service → Repository, garantindo separação de responsabilidades e facilitando manutenção e testes.

2.1 Estrutura de Diretórios

```
ProdutoAPI/  
├── Controllers/  
│   └── ProdutoController.cs    ← Endpoints HTTP  
├── Services/  
│   └── ProdutoService.cs      ← Regras de negócio  
├── Repositories/  
│   └── ProdutoRepository.cs   ← Acesso ao banco
```

└─ Models/	
└─ Produto.cs	← Entidade
└─ ProdutoDtos.cs	← Request/Response
└─ Database/	
└─ DatabaseConnection.cs	← Conexão MySQL
└─ script.sql	← DDL + inserts
└─ Exceptions/	
└─ AppException.cs	← Exceção global
└─ Enums/	
└─ StatusCode.cs	← Códigos HTTP
└─ appsettings.json	← Configurações
└─ Program.cs	← Startup

2.2 Responsabilidades das Camadas

Controller

Recebe as requisições HTTP, valida o modelo de entrada via atributos, delega ao Service e retorna a resposta HTTP correta com o status code apropriado. Não contém lógica de negócio.

Service

Contém todas as regras de negócio: validações de entrada (nome, preço, quantidade), cálculos (valor do estoque), ordenação via LINQ e mapeamento de entidades para DTOs. Lança AppException em caso de violação de regra.

Repository

Responsável exclusivamente pela comunicação com o banco de dados MySQL. Executa as queries parametrizadas (prevenindo SQL Injection) e mapeia os resultados para a entidade Produto.

3. Configuração e Execução

3.1 Pré-requisitos

- SDK .NET 8 instalado (<https://dotnet.microsoft.com/download>)
- MySQL Server 8.0+ em execução
- Visual Studio 2022, VS Code ou qualquer editor C#

3.2 Configuração do Banco de Dados

Execute o script SQL localizado em Database/script.sql no seu cliente MySQL:

```
mysql -u root -p < Database/script.sql
```

Ou copie e cole o conteúdo diretamente no MySQL Workbench / DBeaver.

3.3 Connection String

Edite o arquivo appsettings.json e configure com suas credenciais:

```
"  
  Database=produto_db;User Id=root;Password=SUA_SENHA;"
```

3.4 Executando a API

```
# Restaurar dependências  
dotnet restore  
  
# Executar em modo desenvolvimento  
dotnet run
```

A API estará disponível em <http://localhost:5000>. O Swagger UI será exibido na raiz (/) automaticamente em ambiente de desenvolvimento.

3.5 Instalação das Dependências NuGet

```
dotnet add package MySql.Data --version 9.1.0  
dotnet add package Swashbuckle.AspNetCore --version 6.9.0
```

4. Documentação dos Endpoints

Base URL: `http://localhost:5000/api/produto`

Método	Endpoint	Descrição	Respostas
GET	<code>/api/produto</code>	Listar todos os produtos	200, 500
GET	<code>/api/produto/{id}</code>	Buscar produto por Id	200, 400, 404
POST	<code>/api/produto</code>	Cadastrar novo produto	201, 400, 500
PUT	<code>/api/produto/{id}</code>	Atualizar produto existente	200, 400, 404
DELETE	<code>/api/produto/{id}</code>	Remover produto	204, 400, 404
GET	<code>/api/produto/estoque</code>	Calcular valor total do estoque	200, 500

4.1 GET /api/produto — Listar Produtos

Retorna todos os produtos cadastrados. Aceita parâmetro de ordenação via query string.

Query Parameters:

- `ordenarPor=nome` — ordena por nome (A-Z)
- `ordenarPor=preco` — ordena por preço (menor para maior)
- `ordenarPor=quantidade` — ordena por quantidade em estoque

Exemplo de requisição:

```
GET /api/produto?ordenarPor=preco
```

Resposta de sucesso (200 OK):

```
[
  {
    "id": 1,
    "nome": "Notebook Dell Inspiron 15",
    "preco": 3499.90,
    "quantidade": 10
  }
]
```

4.2 GET /api/produto/{id} — Buscar por Id

Busca um produto específico pelo seu identificador único.

Resposta de sucesso (200 OK):

```
{
```

```
{
  "id": 1,
  "nome": "Notebook Dell Inspiron 15",
  "preco": 3499.90,
  "quantidade": 10
}
```

Resposta de erro (404 Not Found):

```
{
  "statusCode": 404,
  "mensagem": "Produto com Id 99 não encontrado."
}
```

4.3 POST /api/produto — Cadastrar Produto

Cadastra um novo produto no sistema.

Body da requisição (JSON):

```
{
  "nome": "Teclado Mecânico RGB",
  "preco": 299.90,
  "quantidade": 50
}
```

Resposta de sucesso (201 Created):

```
{
  "id": 9,
  "nome": "Teclado Mecânico RGB",
  "preco": 299.90,
  "quantidade": 50
}
```

Resposta de erro (400 Bad Request):

```
{
  "statusCode": 400,
  "mensagem": "O preço deve ser maior que R$ 0,01."
}
```

4.4 PUT /api/produto/{id} — Atualizar Produto

Atualiza todos os campos de um produto existente. O body segue o mesmo formato do POST.

Resposta de sucesso (200 OK): retorna o produto com os dados atualizados.

Resposta de erro (404 Not Found): quando o Id informado não existe no banco.

4.5 DELETE /api/produto/{id} — Remover Produto

Remove permanentemente um produto do sistema.

Resposta de sucesso: HTTP 204 No Content (sem body).

Resposta de erro (404): produto não encontrado.

4.6 GET /api/produto/estoque — Valor Total do Estoque

Calcula o valor total do estoque (soma de preço × quantidade para todos os produtos).

Resposta de sucesso (200 OK):

```
{  
  "totalProdutos": 8,  
  "valorTotal": 97430.50  
}
```

5. Tratamento de Erros

5.1 AppException

Exceção personalizada lançada pelo Service em casos de regra de negócio violada. Carrega o StatusCode correspondente para ser mapeado para o HTTP response correto.

```
public class AppException : Exception
{
    public StatusCode StatusCode { get; }
    public AppException(string message, StatusCode statusCode)
        : base(message) => StatusCode = statusCode;
}
```

5.2 Enum StatusCode

Elimina números mágicos do código. Todos os status HTTP são referenciados via enum:

```
public enum StatusCode
{
    Success      = 200,
    BadRequest   = 400,
    NotFound     = 404,
    InternalError = 500
}
```

5.3 Fluxo de Tratamento no Controller

- AppException capturada → status HTTP do enum + mensagem amigável
- Exception genérica → HTTP 500 + mensagem genérica (detalhe logado no servidor)
- Erros de banco de dados → capturados como Exception genérica

5.4 Tabela de Erros por Validação

Situação	StatusCode	Mensagem
Nome vazio ou nulo	400 BadRequest	O nome do produto é obrigatório.
Nome menor que 2 caracteres	400 BadRequest	O nome deve ter no mínimo 2 caracteres.
Nome maior que 100 caracteres	400 BadRequest	O nome deve ter no máximo 100 caracteres.
Preço menor que 0.01	400 BadRequest	O preço deve ser maior que R\$ 0,01.

Situação	StatusCode	Mensagem
Quantidade negativa	400 BadRequest	A quantidade não pode ser negativa.
Id <= 0	400 BadRequest	O Id deve ser um número positivo.
Produto não encontrado	404 NotFound	Produto com Id {x} não encontrado.
Erro inesperado	500 InternalError	Ocorreu um erro interno. Tente novamente.

6. Validações de Entrada

As validações são realizadas na camada Service, no método ValidarRequest, garantindo que nenhum dado inválido chegue ao banco de dados.

- Nome: obrigatório, entre 2 e 100 caracteres
- Preço: deve ser no mínimo R\$ 0,01 (valores negativos ou zero são recusados)
- Quantidade: não pode ser negativa (zero é aceito para produtos sem estoque)
- Id: deve ser um número inteiro positivo

7. Banco de Dados

7.1 Estrutura da Tabela

```
CREATE TABLE produtos (
    id          INT          NOT NULL AUTO_INCREMENT,
    nome        VARCHAR(100) NOT NULL,
    preco       DECIMAL(10, 2) NOT NULL,
    quantidade  INT          NOT NULL DEFAULT 0,
    PRIMARY KEY (id),
    CONSTRAINT chk_preco      CHECK (preco >= 0.01),
    CONSTRAINT chk_quantidade CHECK (quantidade >= 0)
);
```

7.2 Segurança

- Todas as queries utilizam parâmetros nomeados (@Id, @Nome, @Preco, @Quantidade)
- Prevenção total de SQL Injection via MySqlCommand com parameters
- Conexões abertas e fechadas com await using (IAsyncDisposable)

8. Boas Práticas Aplicadas

- SOLID: Single Responsibility em cada camada, Dependency Injection via construtor
- Sem números mágicos: constantes nomeadas no Service e enum StatusCode
- Records imutáveis para DTOs (ProdutoRequest, ProdutoResponse, EstoqueResponse)
- Async/Await em toda a cadeia de I/O
- Logging estruturado via ILogger (Warning para erros esperados, Error para inesperados)
- Nullable Reference Types habilitado no projeto
- Documentação XML nos endpoints para integração com Swagger

9. Swagger UI

O Swagger UI está disponível na raiz da aplicação (<http://localhost:5000>) em ambiente de desenvolvimento. Permite testar todos os endpoints diretamente pelo navegador, com documentação de request/response e status codes para sucesso e erro.

- Todos os endpoints documentados com summary e response codes
- Exemplos de request body nos schemas
- Status codes de sucesso e erro mapeados com ProducesResponseType